Universidad de San Carlos de Guatemala.

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Ingeniería en Ciencias y Sistemas

Ing. Mario Tobar

**Proyecto de Seminario de Sistemas 1**

**Kei Beauty**

**Manual Tecnico**

Sergio Alejandro Rodriguez López 202131443

Jorge Anibal Bravo Rodríguez 202131782

Quetzaltenango 26 de octubre del 2026

# Índice

**Índice** ..... 2
**1. Diagrama de despliegue** ..... 3
**2. Diagrama de clases del backend** ..... 4
**3. Diagrama de base de datos** ..... 5
**4. Diagramas de procesos (BPMN)** ..... 6
4.1 Acceso y seguridad ..... 6
Activar o desactivar la verificación en dos pasos ..... 6
Reenviar el código de verificación 2FA ..... 7
Recuperación de contraseña ..... 7
4.2 Cliente: catálogo y compra (invitado o registrado) ..... 8
Consultar el catálogo de productos ..... 8
Agregar un producto al carrito ..... 9
Actualizar la cantidad en el carrito ..... 9
4.3 Cliente registrado ..... 10
Agregar un producto a favoritos ..... 10
Consultar y leer notificaciones ..... 10
Consultar el listado de pedidos ..... 11
Consultar el detalle de un pedido (cliente o invitado) ..... 11
Dejar una reseña a un producto ..... 12
Registrar una dirección de envío ..... 12
Alerta de stock (“Avísame cuando haya existencias”) ..... 13
4.4 Administrador ..... 13
Crear un producto ..... 13
Editar un producto ..... 14
Cargar imágenes a la galería ..... 14
Ajustar inventario y alerta de stock ..... 15
Eliminar una categoría ..... 15
Registrar una marca con logo ..... 16
Registrar un descuento ..... 16
Configurar el envío de la tienda ..... 17
Generar reportes de ventas en Excel ..... 17
**Historias de Usuario** ..... 19
**Casos de Uso** ..... 34
**Repositorios de GitHub** ..... 41

# Stack tecnológico

## 1 Backend

El backend es una API REST con Flask 3.0.3 y Python, organizada en Blueprints (uno por recurso). Los paquetes principales, con versiones verificadas en requirements.txt, son:

- Flask-SQLAlchemy 3.1.1: ORM que convierte las tablas en clases Python y viceversa.
- Flask-Migrate 4.0.5 / Alembic: versiona la base de datos en 25 migraciones encadenadas.
- Flask-JWT-Extended 4.6.0: sesiones con token de acceso (24 h), token de refresco (30 días) y token temporal para el 2FA (10 min).
- Flask-CORS 4.0.1: permite que el frontend llame a la API desde su dominio.
- psycopg2-binary 2.9.10 + psycopg (v3) 3.2.9: conectores a PostgreSQL 16.
- Gunicorn 22.0.0: servidor de producción (4 workers).
- imagekitio 5.9.0: subida de fotos de productos, logos y guías a ImageKit.
- openpyxl 3.1.5 / reportlab 4.2.2: exportación de reportes a Excel y PDF.
- pytest 8.3.4: pruebas automatizadas que corren en el pipeline.
- ZohoMail por SMTP: correos de código 2FA, recuperación, confirmación de pedido y avisos.

## 2 Frontend

Aplicación de una sola página (SPA) con Vue 3.4 y Vite 5, con las versiones verificadas en package.json:

- Vue Router 4.3.0: rutas y guardianes (piden login, rol admin o fin del flujo 2FA).
- Pinia 2.1.0: 4 stores (auth, cart, favoritos, notificacion).
- Axios 1.6.0: llamadas a la API con el token inyectado automáticamente.
- Bootstrap 5.3.8 + Bootstrap Icons: diseño responsivo.
- vue-toastification: avisos flotantes (sesión cerrada, errores, confirmaciones).
- chart.js 4.5.1 + vue-chartjs 5.3.4: los 5 gráficos del panel de reportes.
- vitest 3.2.4: pruebas del frontend que corren en el pipeline (npm test).

## 3 Infraestructura

- AWS: S3 + CloudFront (frontend), EC2 Ubuntu con nginx + Gunicorn en Docker (backend), RDS PostgreSQL 16 privada (datos). Todo con HTTPS.
- Terraform (iac/terraform/): réplica versionada de esa infraestructura (red, cómputo, base de datos, frontend e IAM).
- GitHub Actions: pipeline de pruebas y despliegue en cada push a main.
- Docker + docker-compose.prod.yml: la API corre contenerizada en el servidor.

## 4 Nuevas tecnologías con justificación

| **Tecnología** | **Para qué se usa** | **Por qué elegimos esta** |
|---|---|---|
| AWS (S3, CloudFront, EC2, RDS) | Alojar frontend, backend y base de datos en la nube | Es el estándar de la industria: hay más documentación, más profesionales que pueden retomar el proyecto y ofrece alta disponibilidad. Su capa gratuita permite probar el despliegue en cuentas nuevas. (Matiz honesto: la capa gratuita alcanza para probar, no para producción sostenida.) |
| reportlab 4.2.2 | Exportar los 7 reportes a PDF (?pdf=1) | Genera el PDF en memoria sin dependencias del sistema (a diferencia de weasyprint, que exige librerías nativas), ideal para un contenedor liviano. |

| | | |
|---|---|---|
| chart.js + vue-chartjs | 5 gráficos del panel de reportes | Integración directa con Vue y suficiente para barras y líneas; alternativas como apexcharts pesan más para lo que se necesita. |
| pytest + vitest | Pruebas que bloquean el deploy si fallan | Son los corredores estándar de cada ecosistema (Python y Vite); el pipeline exige ambos en verde antes de desplegar. |
| Terraform | Réplica versionada de la infraestructura | Declara red, cómputo, BD, S3, CloudFront e IAM en código revisable, en vez de clics manuales en consola que nadie puede auditar. Se versiona como réplica observada: no se aplica sin revisión porque los recursos ya existen. |
| psycopg v3 | Conector PostgreSQL que SQLAlchemy 2 exige por defecto | Sin él, la app no arranca con las versiones actuales; convive con psycopg2-binary heredado. |

## 2. Comparativa Fase 2 con  Fase 3

**Infraestructura.** En Fase 2 todo corría en la PC local con Docker Compose y se exponía con un túnel ngrok de un solo origen (la API servía también el frontend). En Fase 3 cada pieza vive en AWS: frontend desacoplado en S3 + CloudFront, backend en EC2 y base de datos en RDS privada, con HTTPS real y despliegue automático. Se eliminó ngrok y el modo de origen único.

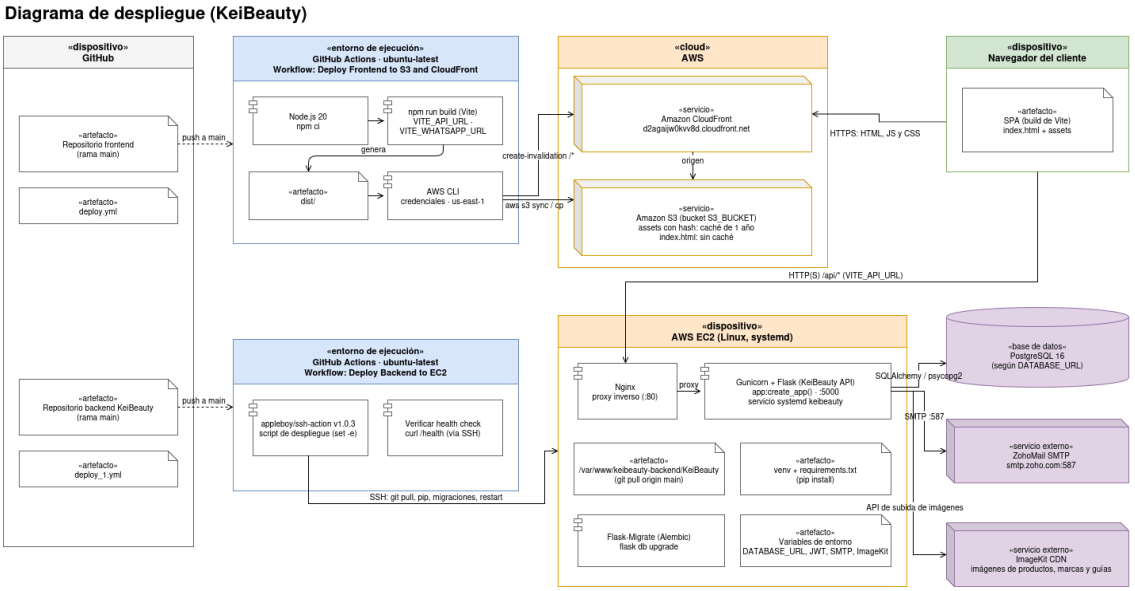
**Seguridad y autenticación.** El 2FA ya existía y se mantiene (código de 6 dígitos, 5 minutos, 5 intentos). Lo que cambió es la recuperación de contraseña: antes era un enlace con token por correo (24 h); ahora es un código de 6 dígitos en 3 pasos (pedir, verificar, definir), con expiración de 15 minutos y un solo uso. Además el registro duplicado devuelve 409 con código EMAIL_DUPLICADO y el perfil permite activar/desactivar el 2FA con interruptor.

**Datos.** La Fase 2 terminó en la migración 2026_inventario_costo_unitario con 4 catálogos. La Fase 3 agrega 6 migraciones: códigos de recuperación, cascadas al borrar usuario, direcciones y teléfonos múltiples, configuración de tienda, descuentos por producto y bandera de envío gratis. El pedido ahora guarda subtotal, costo de envío, precio real y descuento aplicado.

Reportes. Antes solo Excel en 6 reportes (y ventas-por-mes ignoraba el filtro de fechas). Ahora son 7 reportes (nuevo: ventas con descuento con comparación contra el periodo anterior), todos con Excel y PDF, subtítulo de periodo, filtro por rango respetado y 5 gráficos en el panel.

Frontend. Wizard de recuperación en 3 pasos, aviso de "contraseña restablecida" al volver al login, toast al cerrar sesión, galería visible al editar productos, vista de configuración de envío, buscador centrado, barra de filtros, cuadrícula 4/3/2 y panel admin agrupado. Además dist/ dejó de versionarse (lo genera el pipeline).

# 1. Diagrama de despliegue



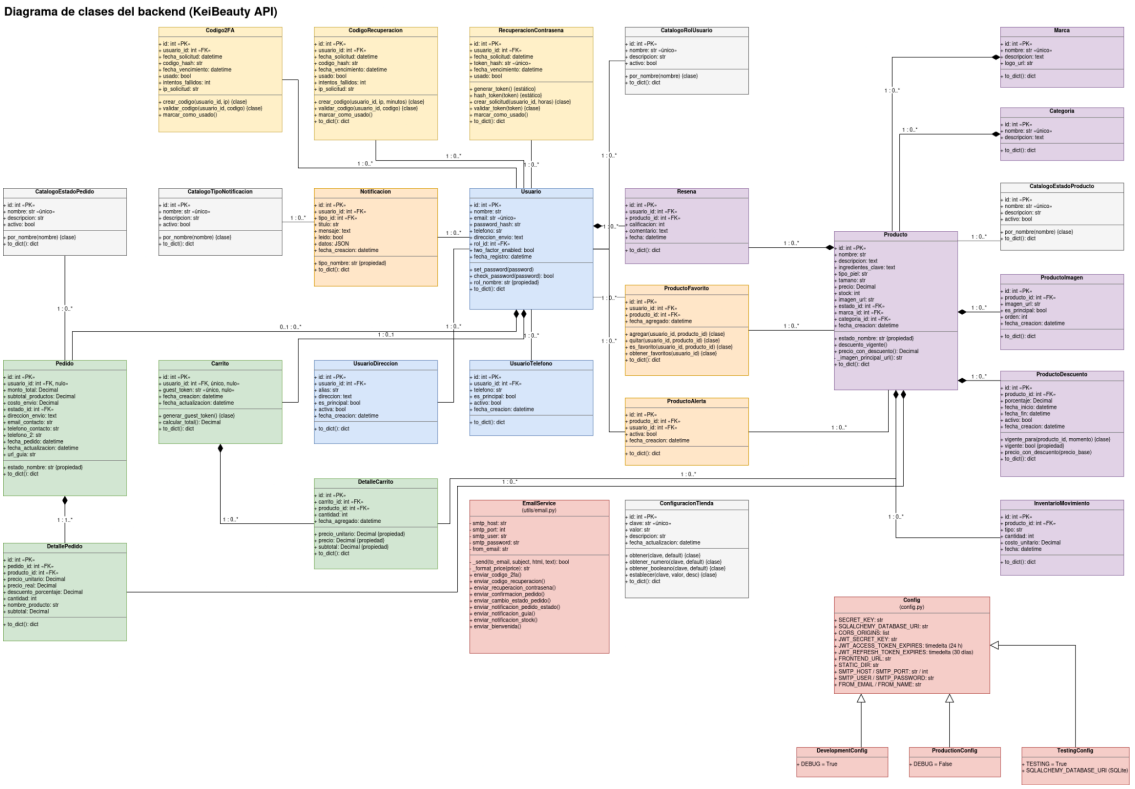

La tienda funciona con tres piezas en la nube de AWS. El navegador del cliente recibe la página (HTML, JS y CSS) desde CloudFront, que a su vez la toma de un bucket S3 privado: nadie accede al bucket directamente, solo CloudFront puede leerlo. Cuando la página necesita datos (productos, pedidos, login), llama por internet a la API que vive en una máquina EC2 con Linux: ahí el nginx recibe las peticiones y se las pasa a Gunicorn + Flask. La API guarda todo en PostgreSQL 16 (RDS), que no es pública: solo la máquina EC2 puede conectarse a ella. Para los correos (códigos de acceso y avisos) usa ZohoMail por SMTP, y para las fotos de productos, marcas y guías usa ImageKit.

**Ficha técnica:**

cada vez que se sube código a la rama main, GitHub Actions despliega solo: el frontend se compila con Vite, se sube a S3 y se purga la caché de CloudFront; el backend se actualiza por SSH, instala dependencias, aplica las migraciones con flask

db upgrade y verifica el /health antes de dar por terminado el despliegue. Las claves y URLs viven en variables de entorno, nunca en el código.

## 2. Diagrama de clases del backend

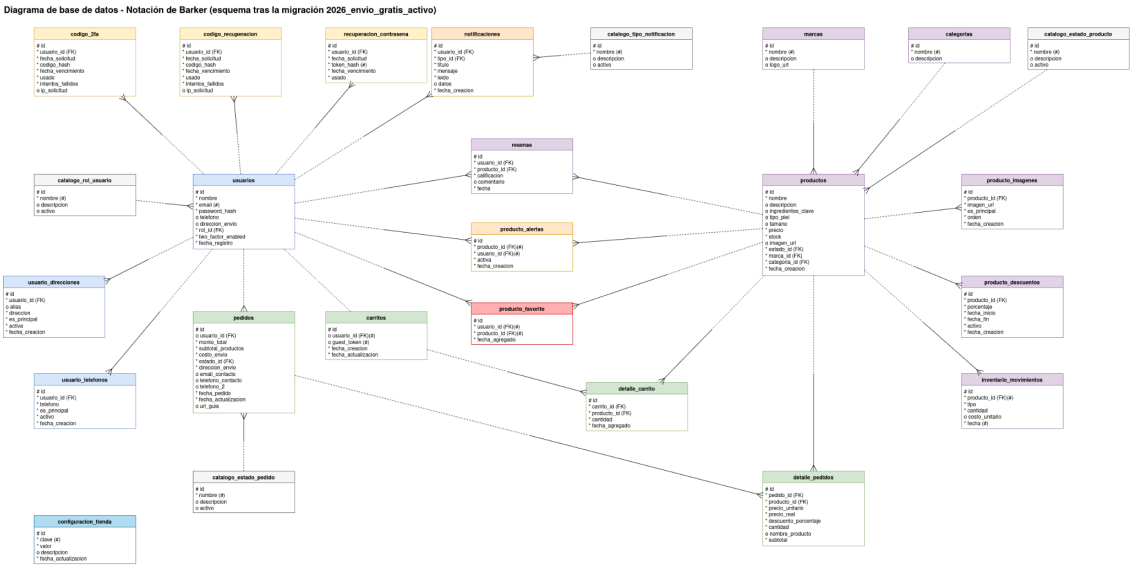



Es el mapa de las clases del API (Flask + SQLAlchemy). Al centro está Usuario (datos, contraseña con hash, rol y 2FA), relacionado con sus direcciones, teléfonos, pedidos, carrito, favoritos, reseñas, notificaciones y códigos de seguridad. A la derecha, el bloque comercial: Producto (con marca, categoría, imágenes de galería, descuentos por fecha y movimientos de inventario) y Pedido con su detalle (precio real cobrado y descuento aplicado). Arriba, las clases de seguridad (Codigo2FA, CodigoRecuperacion, RecuperacionContrasena) y los catálogos (roles, estados y tipos, que evitan textos sueltos en la base). Abajo, las utilidades: EmailService (todos

los correos que envía la tienda), ConfiguracionTienda (costo de envío y mínimo para envío gratis) y la configuración por ambiente (desarrollo, producción y pruebas).

**Ficha técnica:** cada modelo expone un método to_dict() que convierte el objeto a JSON, y los nombres legibles (rol, estado, tipo) se resuelven siempre contra las tablas catálogo. La autenticación usa tokens JWT con un token temporal para el flujo de 2FA.

## 3. Diagrama de base de datos




Muestra todas las tablas en notación Barker tras la última migración. Se lee por grupos: **usuarios** al centro con sus tablas de seguridad (códigos 2FA, códigos de recuperación, solicitudes de contraseña) y notificaciones; **usuario_direcciones** y **usuario_telefonos** para las varias direcciones y teléfonos por cuenta; **productos** con marcas, categorías, imágenes, descuentos, alertas, favoritos, reseñas e inventario; **pedidos** con su detalle (subtotal, costo de envío, precio real y descuento) y su catálogo de estados; **carritos** con su detalle (incluye compras de invitados por guest_token); y **configuracion_tienda** con las claves de la política de envío.

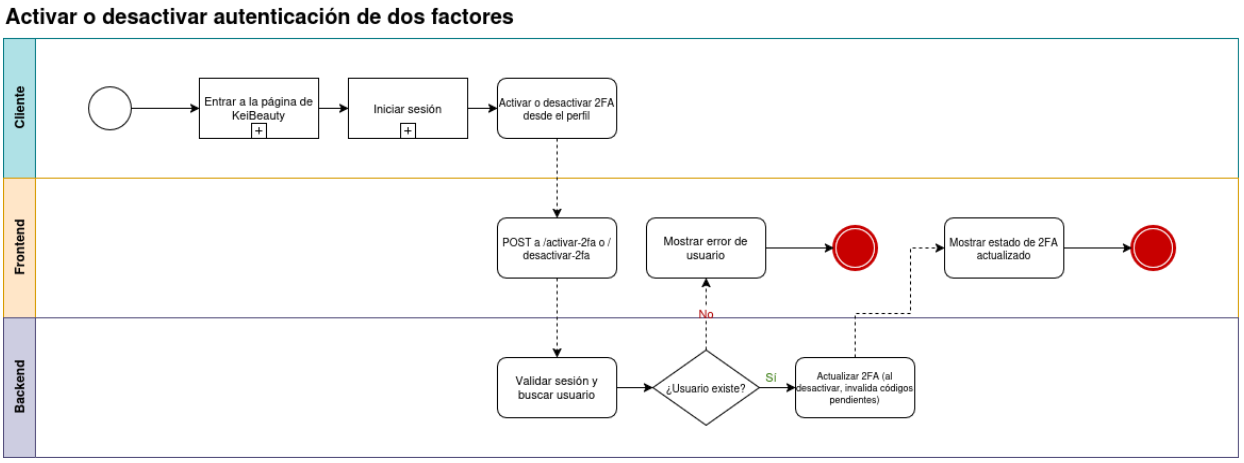
Ficha técnica: el esquema evoluciona con **Alembic** (**Flask-Migrate**): migraciones reversibles en cadena lineal (un solo head), llaves foráneas con nombre y borrado en cascada hacia usuarios, y columnas nuevas agregadas primero como opcionales con relleno de datos antes de volverlas obligatorias.

# 4. Diagramas de procesos (BPMN)

Todos los diagramas usan tres carriles: lo que hace la persona (**Cliente** o **Admin**), lo que hace la página (**Frontend**) y lo que hace el servidor (**Backend**); algunos agregan el carril del servicio externo (**Email** o **ImageKit**). Los rombos verdes/rojos son decisiones: por el "Sí" avanza el flujo, por el "No" muestra el error correspondiente.

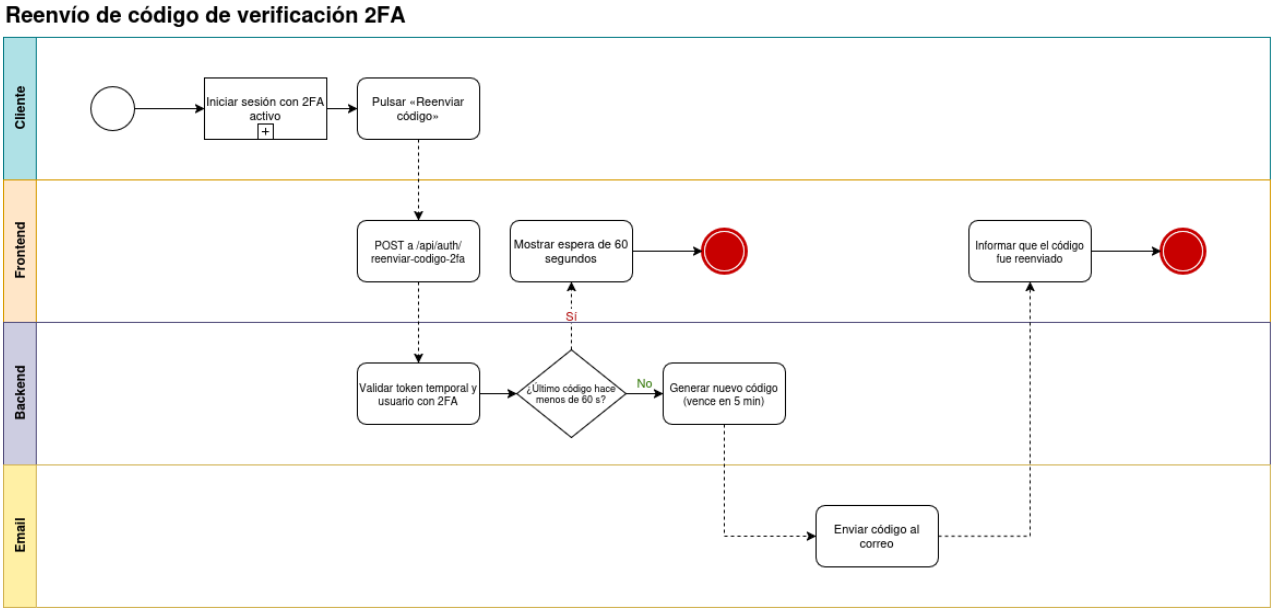
## 4.1 Acceso y seguridad

### Activar o desactivar la verificación en dos pasos



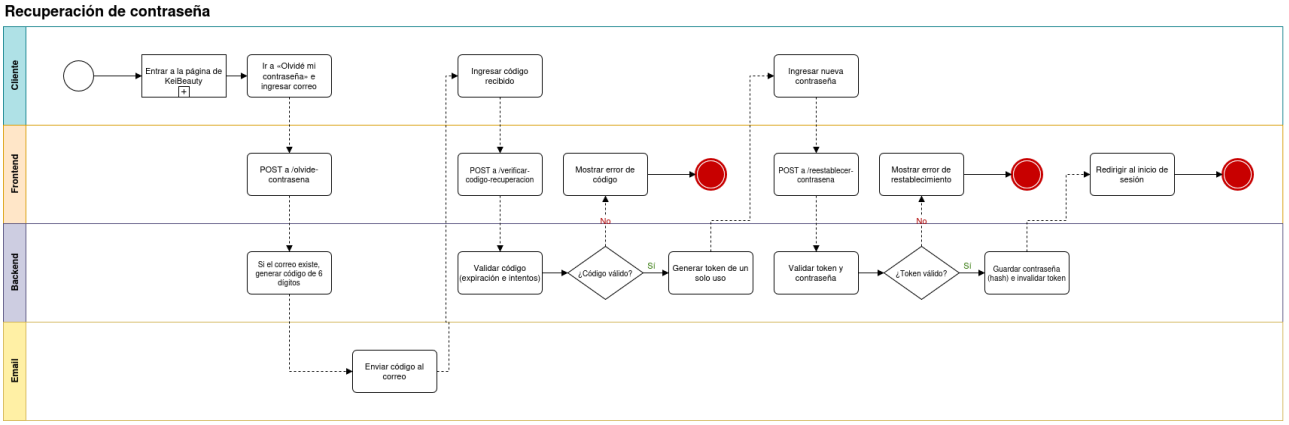

Desde su perfil, el cliente con sesión iniciada cambia el interruptor de 2FA. La página llama a activar o desactivar, el servidor valida la sesión, actualiza la bandera del usuario (y al desactivar invalida los códigos pendientes) y la página muestra el estado actualizado.

## Reenviar el código de verificación 2FA




Si el código de acceso no llegó al correo, el cliente lo pide de nuevo. El servidor valida el token temporal y revisa que hayan pasado más de 60 segundos desde el último envío: si fue hace menos, la página muestra la espera; si no, genera un código nuevo (vence en 5 minutos), lo envía al correo por el carril Email y confirma el reenvío.

## Recuperación de contraseña

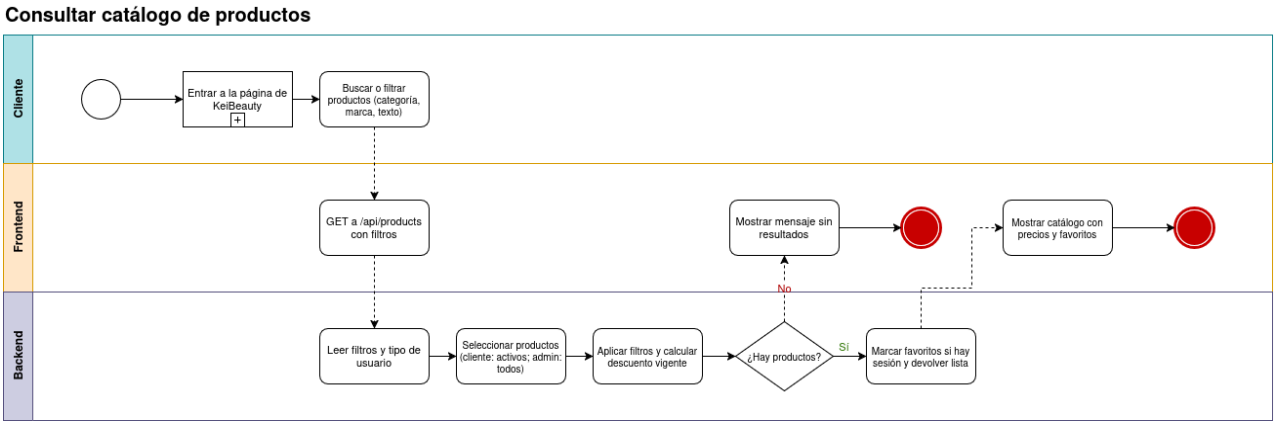



Proceso en tres pasos: el cliente pide el código con su correo (el servidor responde siempre igual, exista o no la cuenta, y envía el código solo si existe), luego ingresa el

código recibido (se valida expiración e intentos; si es válido se crea un token de un solo uso) y por último define la nueva contraseña (se valida el token, se guarda el hash y se invalida el token). Al final se redirige al inicio de sesión.

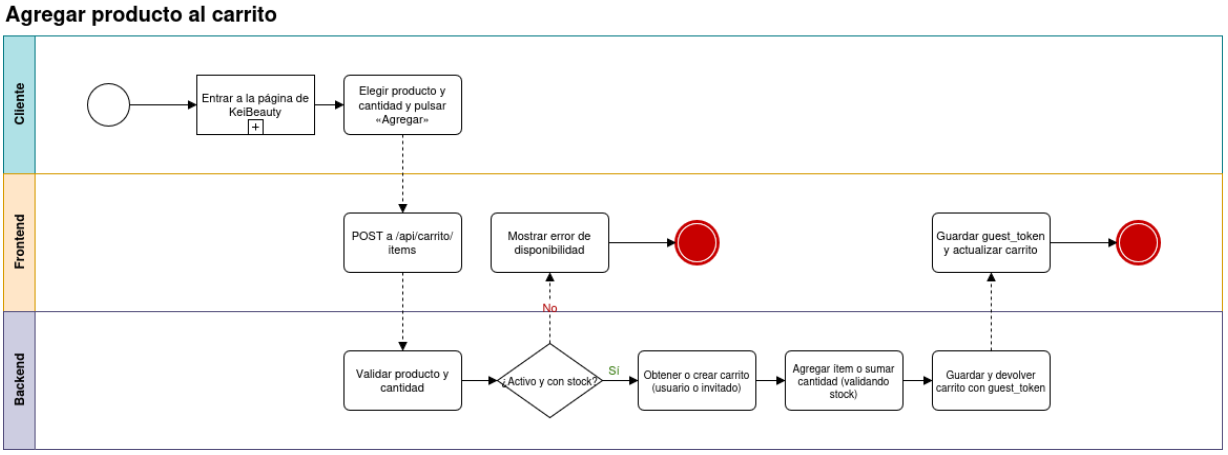
## 4.2 Cliente: catálogo y compra (invitado o registrado)

### Consultar el catálogo de productos



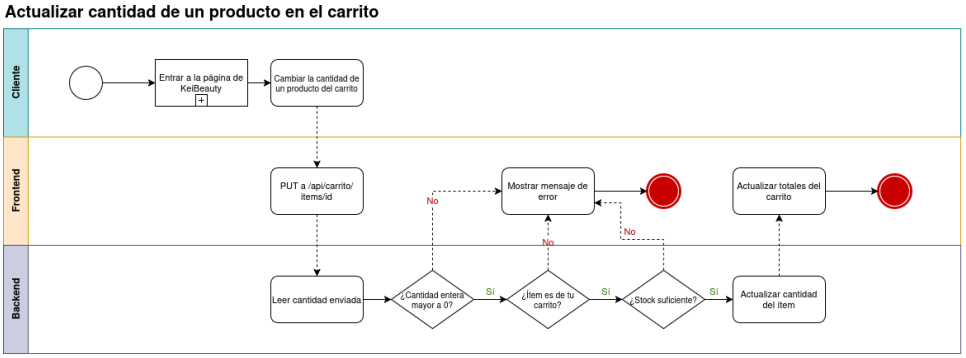

El cliente entra a la tienda y busca o filtra por categoría, marca o texto. La página pide GET /api/products con los filtros; el servidor lee el tipo de usuario (los visitantes solo ven productos activos, el admin los ve todos), aplica los filtros, calcula el descuento vigente de cada producto y marca favoritos si hay sesión. Si no hay resultados, se muestra el mensaje correspondiente.

## Agregar un producto al carrito




El cliente elige producto y cantidad y pulsa "Agregar". La página envía POST /api/carrito/items; el servidor valida que el producto esté activo y con stock, obtiene o crea el carrito (del usuario o del invitado por guest_token), suma la cantidad validando existencias y lo devuelve. Si no hay stock, se muestra el error de disponibilidad.

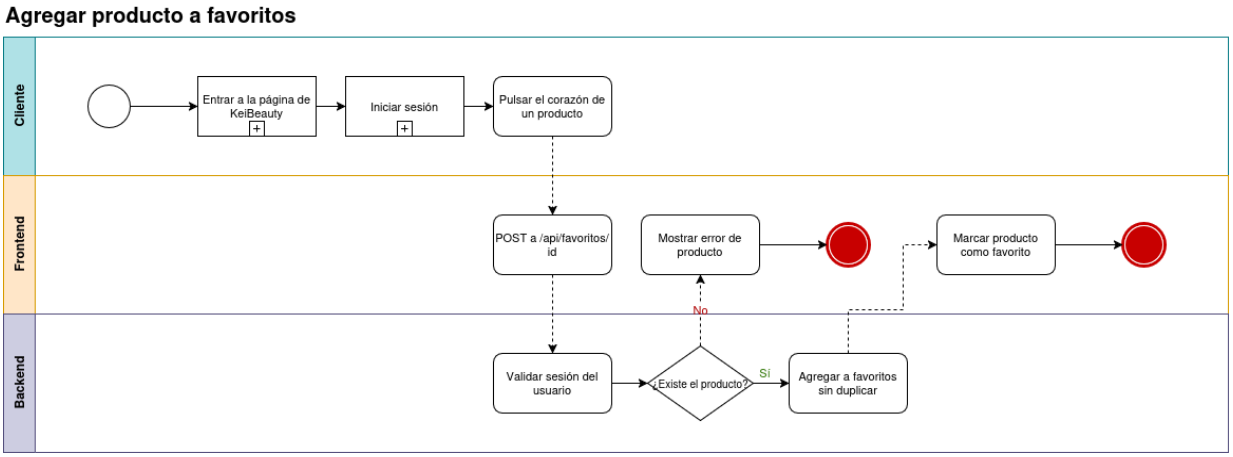
## Actualizar la cantidad en el carrito




Al cambiar la cantidad de un ítem, la página envía PUT /api/carrito/items/id. El servidor valida que la cantidad sea un entero mayor a cero, que el ítem pertenezca a ese carrito y que haya stock suficiente; solo entonces actualiza y la página recalcula los totales. Cualquier fallo muestra su mensaje de error.

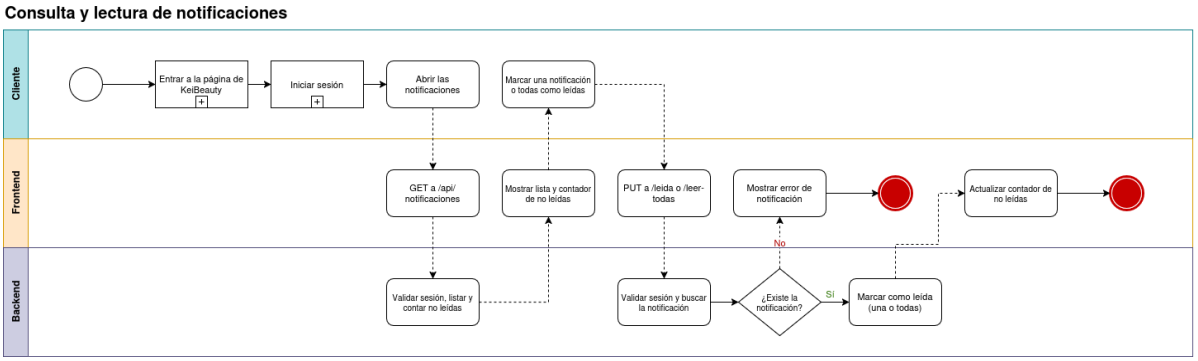
## 4.3 Cliente registrado

### Agregar un producto a favoritos



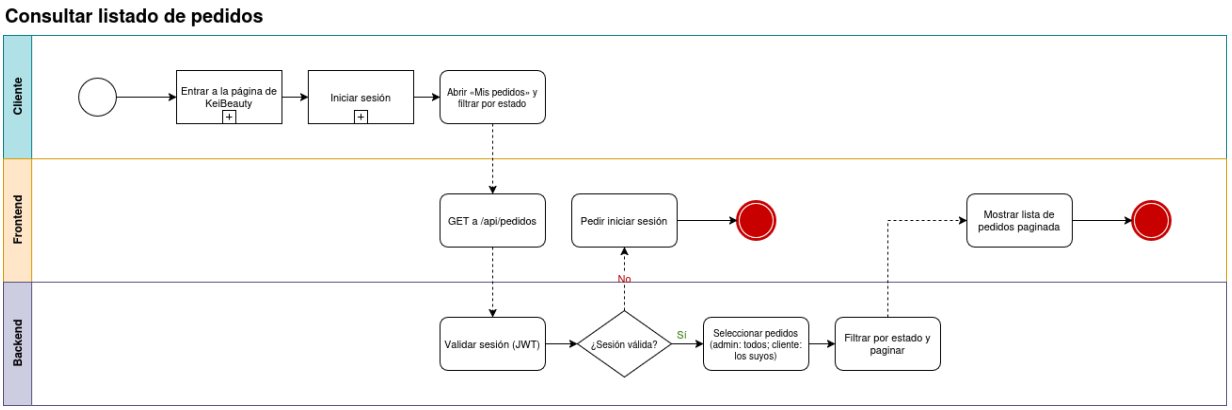

Con sesión iniciada, el cliente pulsa el corazón de un producto. La página llama a POST /api/favoritos/id; el servidor valida la sesión y que el producto exista, y lo guarda sin duplicar. El corazón queda marcado en la tienda.

### Consultar y leer notificaciones



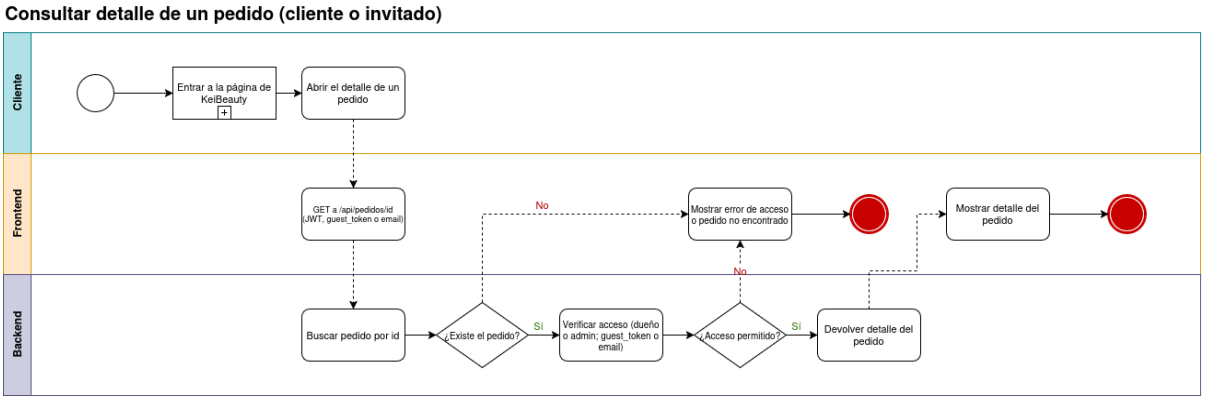

El cliente abre la campana de notificaciones (cambios de estado de pedidos, guías agregadas, stock recuperado). La página pide la lista con su contador de no leídas; al marcar una o todas como leídas, el servidor valida la sesión y actualiza el estado, y el contador se pone al día.

## Consultar el listado de pedidos



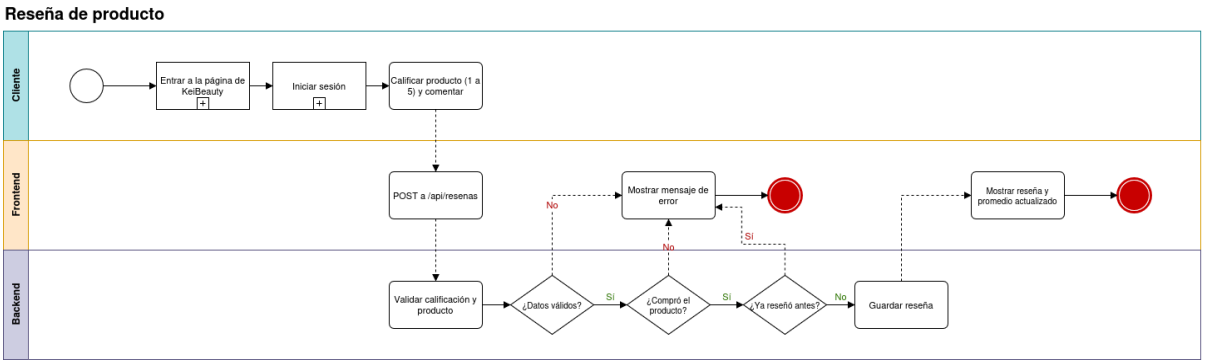

En "Mis pedidos" el cliente puede filtrar por estado. La página pide GET /api/pedidos con el JWT; sin sesión válida pide iniciar sesión. El servidor devuelve los pedidos del cliente (o todos si es admin), filtrados por estado y paginados, y la página muestra la lista.

## Consultar el detalle de un pedido (cliente o invitado)



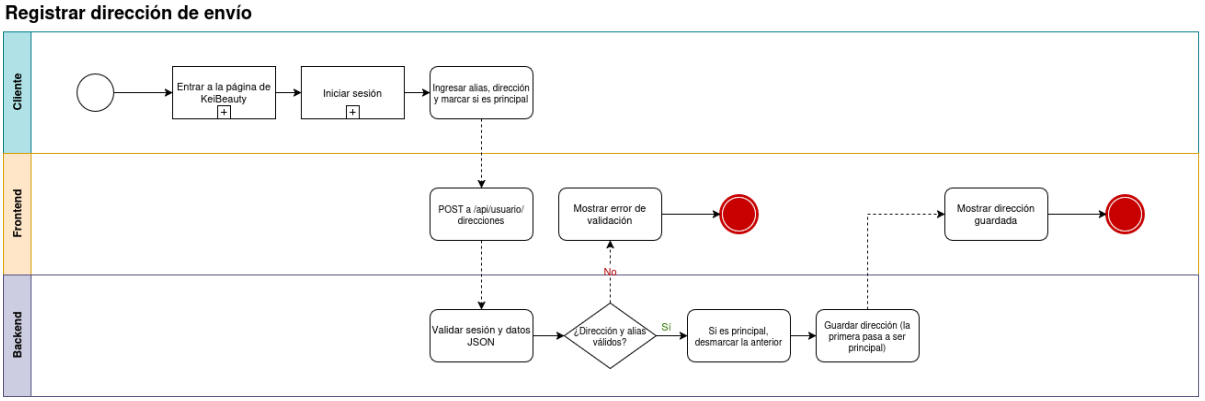

Al abrir un pedido, el servidor primero verifica que exista y luego el permiso: el dueño, un admin, o el invitado con su guest_token o email de contacto. Solo con acceso permitido devuelve el detalle completo (productos, desglose, guía); si no, muestra error de acceso o de pedido no encontrado.

## Dejar una reseña a un producto



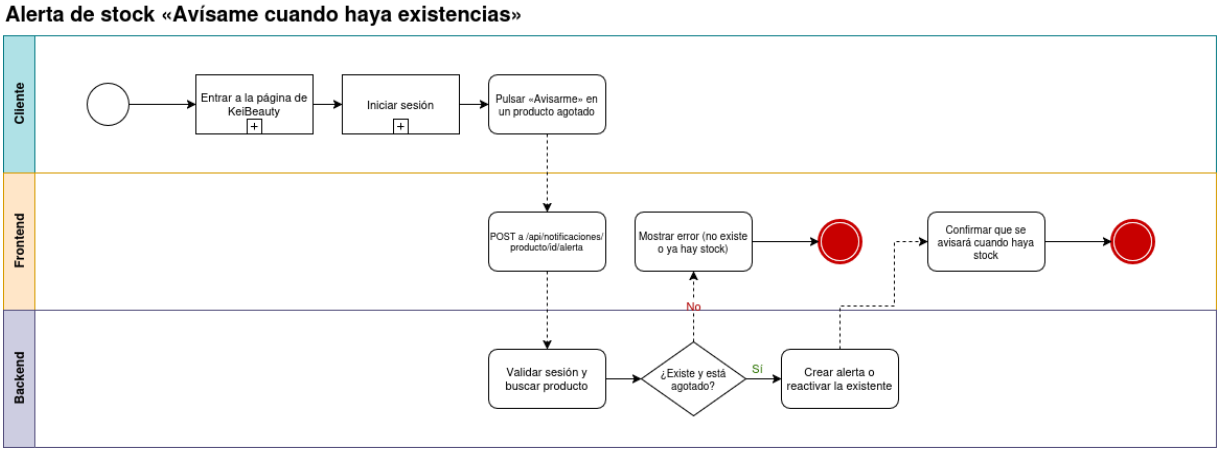

El cliente califica de 1 a 5 estrellas y comenta. El servidor valida los datos, exige haber comprado el producto y que no lo haya reseñado antes; solo entonces guarda la reseña y la página muestra la reseña con el promedio actualizado.

## Registrar una dirección de envío




Desde el perfil, el cliente ingresa alias, dirección y si será la principal. El servidor valida la sesión y los datos; si la marca como principal, desmarca la anterior (la primera dirección guardada queda como principal automáticamente). La página muestra la dirección guardada en la lista.

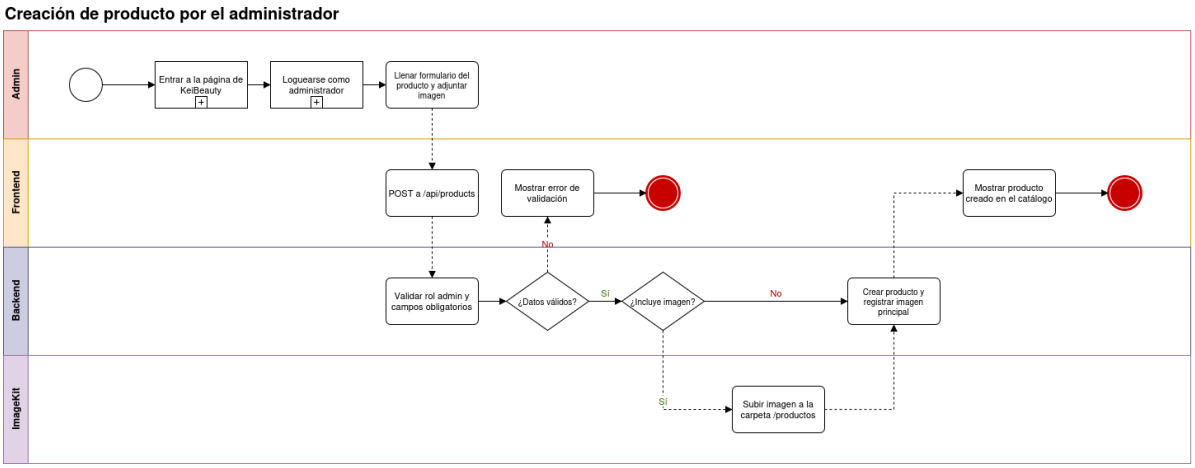
### Alerta de stock (“Avísame cuando haya existencias”)




En un producto agotado, el cliente pulsa “Avísame”. El servidor valida la sesión y que el producto exista y siga agotado (si ya volvió el stock, avisa el error), y crea o reactiva la alerta. Cuando el admin reponga existencias, el cliente recibe el aviso en la app y por correo.

## 4.4 Administrador

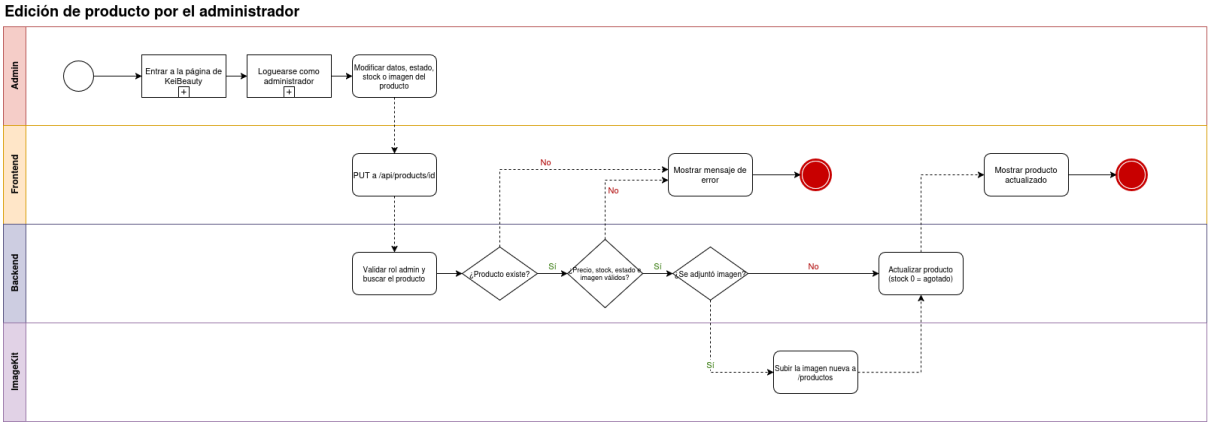
### Crear un producto




Con sesión de administrador, completa el formulario y adjunta imagen si quiere. El servidor valida el rol y los campos obligatorios; si hay imagen, primero la sube a

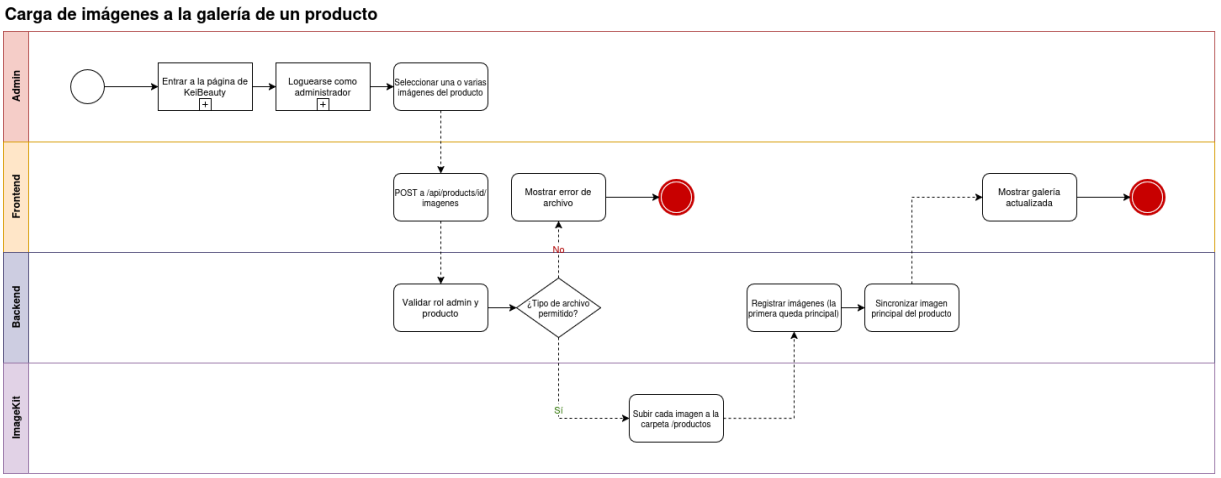
ImageKit (carpeta /productos) y luego crea el producto registrando la imagen como principal. El producto aparece en el catálogo.

### Editar un producto




El admin modifica datos, estado, stock o imagen. El servidor valida el rol, que el producto exista y que precio, stock e imagen sean válidos; si se adjuntó imagen nueva, la sube a ImageKit antes de actualizar. Si el stock queda en cero, el producto pasa a estado agotado automáticamente.

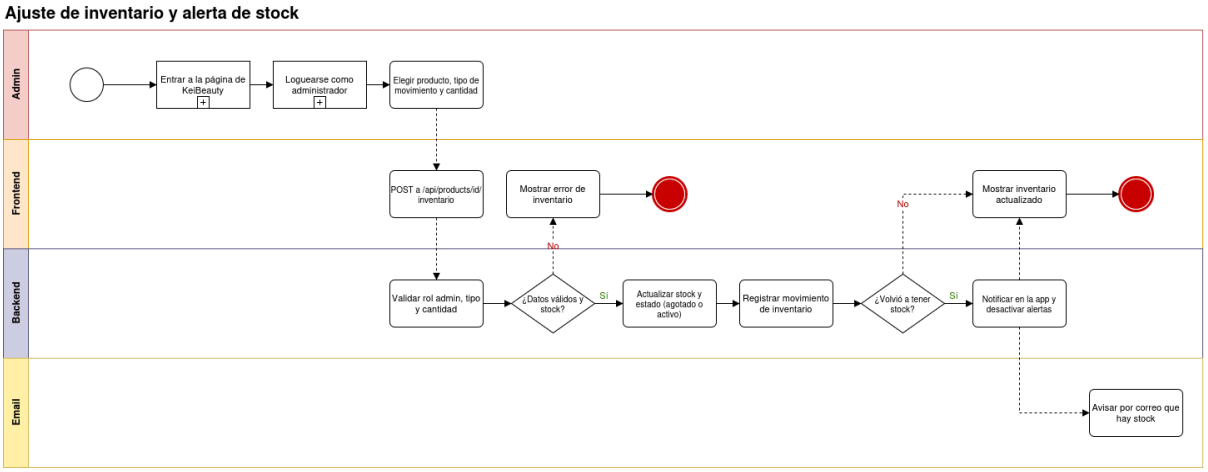
### Cargar imágenes a la galería




El admin selecciona una o varias imágenes del producto. El servidor valida el rol, el producto y que cada archivo sea de un tipo permitido; sube cada una a ImageKit, las

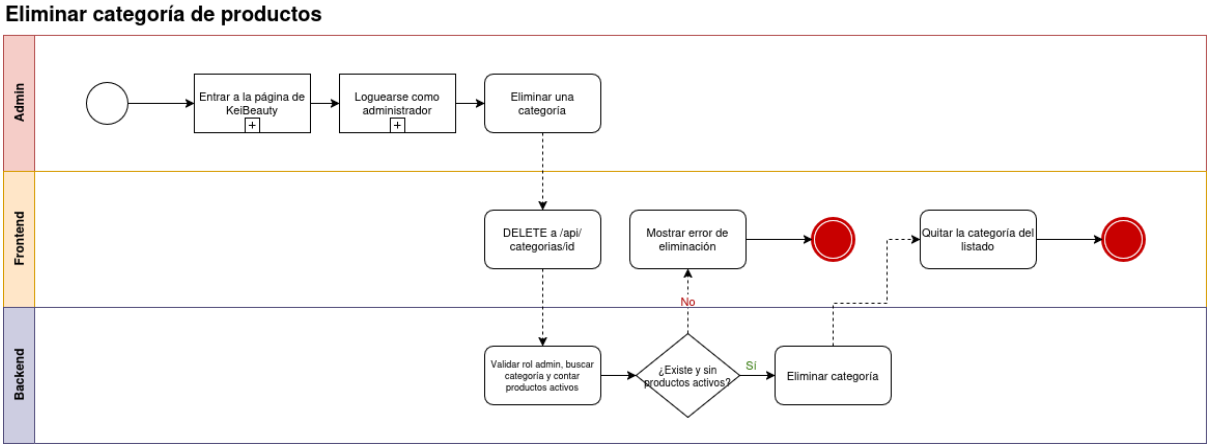
registra (la primera queda como principal) y sincroniza la imagen principal del producto. La galería se muestra actualizada.

### Ajustar inventario y alerta de stock



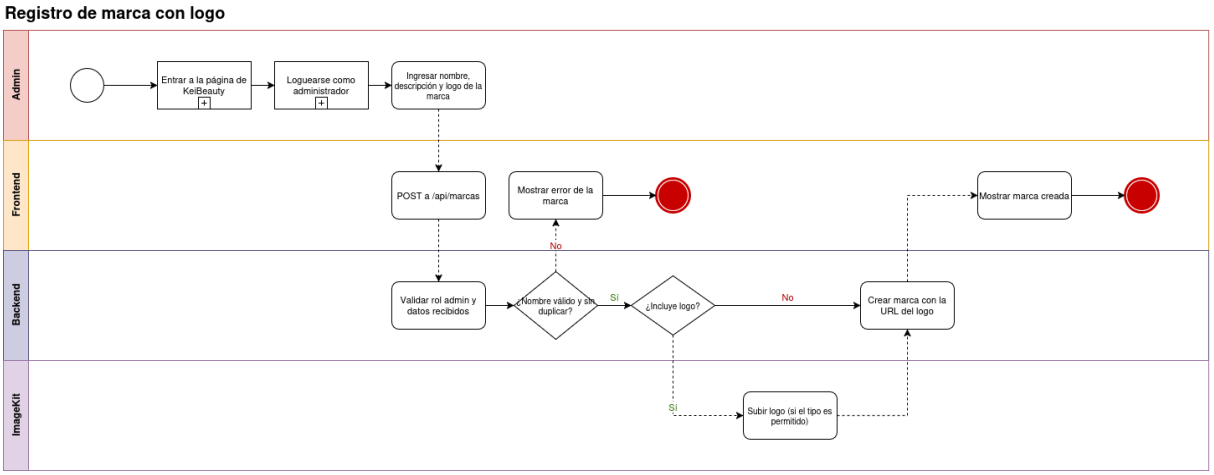

El admin elige producto, tipo de movimiento (entrada o salida) y cantidad. El servidor valida todo, actualiza el stock y su estado (agotado o activo) y registra el movimiento. Si el producto volvió a tener stock, notifica en la app a quienes pidieron el aviso y desactiva esas alertas, además de avisar por correo que ya hay existencias.

### Eliminar una categoría



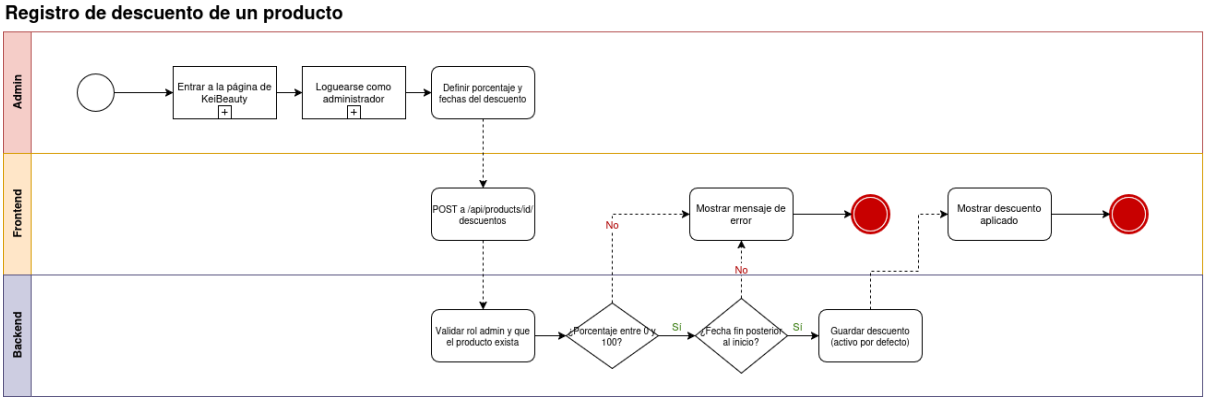
El admin elimina una categoría con DELETE /api/categorias/id. El servidor valida el rol y cuenta los productos activos de esa categoría: solo la elimina si existe y no tiene productos activos; si tiene, muestra el error y la categoría se queda en el listado.

### Registrar una marca con logo



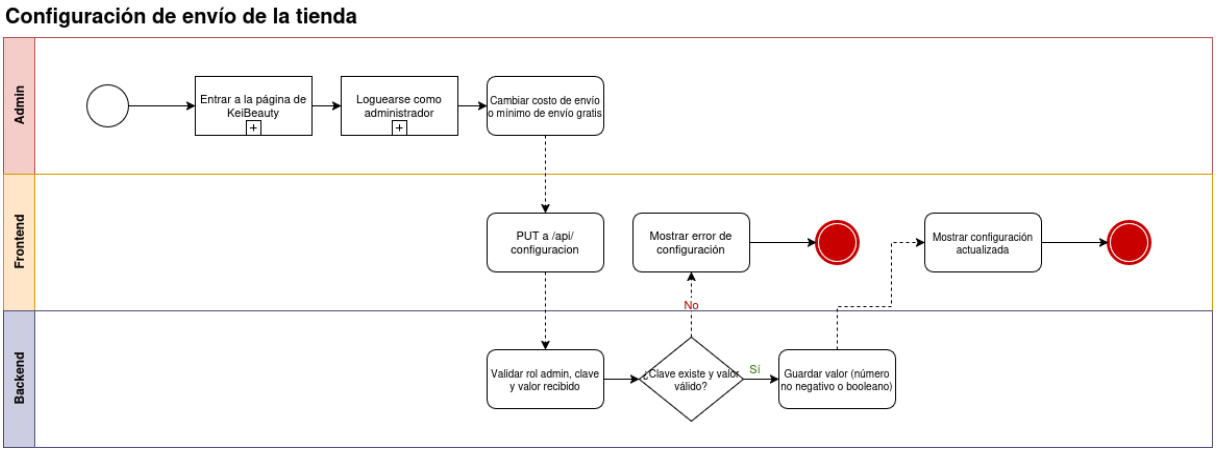

El admin ingresa nombre, descripción y logo de la marca. El servidor valida el rol y que el nombre sea válido y no esté duplicado; si incluye logo y el tipo de archivo es permitido, lo sube a ImageKit y crea la marca con la URL del logo. La marca aparece creada en el listado.

### Registrar un descuento



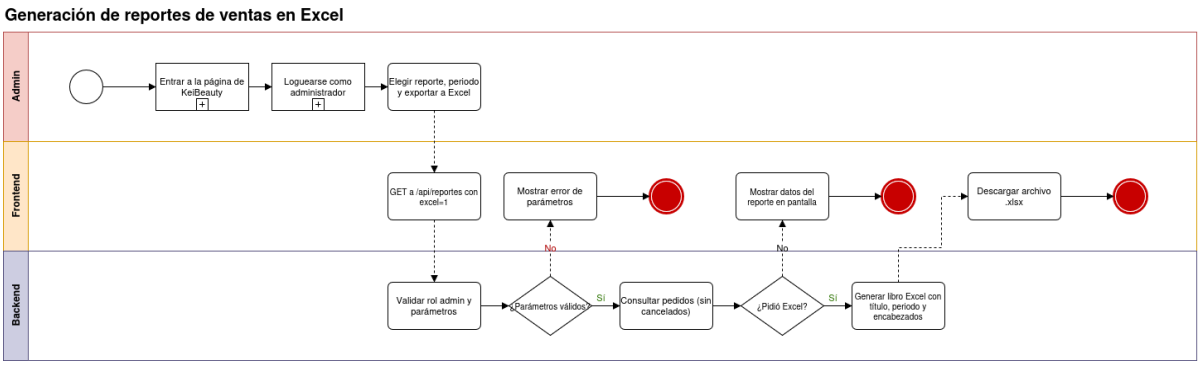
El admin define el porcentaje y las fechas del descuento. El servidor valida el rol, que el producto exista, que el porcentaje esté entre 0 y 100 y que la fecha de fin sea posterior a la de inicio; entonces guarda el descuento (activo por defecto) y la tienda muestra el precio con descuento mientras esté vigente.

### Configurar el envío de la tienda




El admin cambia el costo de envío o el mínimo para envío gratis. El servidor valida el rol y que la clave exista y el valor sea válido (número no negativo o booleano para el interruptor); lo guarda y la página muestra la configuración actualizada, que aplica de inmediato al carrito y al checkout.

### Generar reportes de ventas en Excel



El admin elige el reporte, el periodo y la exportación a Excel. La página pide GET /api/reportes con excel=1; el servidor valida el rol y los parámetros, consulta los pedidos (sin cancelados) y, si se pidió Excel, genera el libro con título, periodo y encabezados para descargar el .xlsx. Los datos también se muestran en pantalla con sus gráficos.

# 3. Modelado de datos

Las tablas se agrupan así (ver figura diagrama_tablas en el documento de diagramas):

- **Personas y seguridad**: usuarios al centro, con codigo_2fa, codigo_recuperacion, recuperacion_contrasena y notificaciones. usuario_direcciones y usuario_telefonos guardan las N direcciones y teléfonos por cuenta (una sola principal por tipo, sincronizada con las columnas heredadas de usuarios).
- **Catálogos**: catalogo_rol_usuario, catalogo_estado_producto, catalogo_estado_pedido, catalogo_tipo_notificacion. Ningún estado/rol/tipo vive como texto suelto: todo es llave foránea.
- **Comercio**: productos (con marca_id, categoria_id, estado_id), producto_imagenes (galería con principal), producto_descuentos (porcentaje + rango de fechas), inventario_movimientos, producto_alertas, producto_favorito, resenas.
- **Compra**: carritos + detalle_carrito (incluye invitados por guest_token), pedidos + detalle_pedidos.
- **Configuración**: configuracion_tienda (clave/valor: costo_envio, minimo_envio_gratis, envio_gratis_activo).

Decisiones clave: (1) catálogos en vez de strings sueltos para integridad referencial; (2) borrado en cascada hacia usuarios en 6 tablas hijas (a nivel ORM y con ON DELETE CASCADE); (3) detalle_pedidos guarda precio_unitario (catálogo), precio_real (cobrado) y descuento_porcentaje para trazabilidad; (4) el pedido guarda foto (snapshot) de dirección, teléfonos, subtotal y envío al momento de la compra.

# 4. Migraciones del modelo

Hay **25 migraciones** en cadena lineal con **un solo head** (2026_envio_gratis_activo), verificable con flask db heads. La convención es: columna nueva primero opcional → relleno de datos (backfill) → obligatoria; llaves foráneas siempre con nombre; cada migración con su reversa probada (upgrade/downgrade).

Oleadas Fase 3 (en orden): 2026_codigo_recuperacion → 2026_cascade_usuarios_fk → 2026_direcciones_telefonos → 2026_configuracion_tienda → 2026_producto_descuentos → 2026_envio_gratis_activo (más 2026_subtotal_detalle_pedidos y 2026_email_telefono_pedido del flujo de pedidos).

# 5. Seguridad avanzada

- **2FA**: código de 6 dígitos por correo, hasheado en codigo_2fa, expira en 5 minutos, máximo 5 intentos, reenvío con espera de 60 s, token temporal de 10 min durante el flujo.

- **Recuperación**: código de 6 dígitos (no enlace), expira en 15 minutos, un solo uso; al verificarse emite el token de restablecimiento, que también es de un solo uso. Respuesta siempre 200 para no revelar qué correos existen.
- **Secretos**: viven en GitHub Secrets (CI) y en el .env del EC2 (DATABASE_URL interna, JWT, SMTP, ImageKit). Nada en git: verificado que ningún .env, .pem o clave está versionado.
- **Contraseñas y sesiones**: hash con werkzeug (nunca texto plano), sesiones JWT (acceso 24 h, refresco 30 días), decoradores por rol (admin_required) y rechazo de tokens temporales a medio flujo 2FA.
- **Cifrado**: TLS en tránsito (HTTPS en ambos dominios) y storage_encrypted=true en la RDS.

# 6. HTTPS e IAM

- **HTTPS**: certificados Let's Encrypt vigentes en keibeauty-api.duckdns.org y CloudFront; HTTP redirige a HTTPS (301) y el frontend no carga contenido mixto (verificado: cero referencias http://).
- **IAM**: existe el usuario CI github-actions-frontend (hoy con permisos amplios S3/CloudFront: migrar a la política acotada de iac/terraform/iam.tf); **no existe** usuario CI para el backend (despliega por SSH, sin políticas AWS). El usuario humano shjn figura con acceso directo de administrador, sin MFA y sin haber iniciado sesión: antes de usarlo hay que probar el login, activar MFA y recortar permisos (o eliminarlo). No se usa la cuenta root.

# 7. Estrategia CI/CD

En cada push a main, por repositorio: primero el job test (pytest en backend, vitest en frontend) y solo si está en verde corre el deploy (needs: test). El backend actualiza por SSH, reconstruye el contenedor, aplica flask db upgrade dentro del contenedor (si la migración falla, set -e detiene todo y el sitio sigue con la versión anterior) y verifica el /health. El frontend compila, sube a S3 (assets con hash cacheados un año, index.html sin caché) e invalida CloudFront. develop no despliega: es solo integración.

# 8. Estructura de paquetes

**Backend** (KeiBeauty/): app.py (fábrica create_app()), config.py (ambientes), seed.py (datos iniciales + configuración de envío). models/ (21 modelos: los de Fase 2 más codigo_recuperacion, usuario_direccion —con ambas clases, direcciones y teléfonos—, producto_descuento y configuracion_tienda). routes/ (12 blueprints: los 10 de Fase 2 más usuario y configuracion). utils/ (decoradores y correos, incluido el de código de recuperación). migrations/versions/ (25 archivos). tests/ (pytest). iac/terraform/ (red, cómputo+BD, frontend, IAM, variables).

**Frontend** (KeiBeauty-frontend/): 25 vistas (las de Fase 2 más AdminConfiguracionView), 4 stores Pinia, services/api.js, composable useEnvio.js (desglose de envío compartido entre carrito y checkout) y componente NotificacionesLista.vue. Pruebas en tests/ con vitest.

# Historias de Usuario

**HU-01 — Registro de cliente**

| | |
|---|---|
| **Rol / Actor** | Invitado |
| **Narrativa** | Como visitante, quiero crear una cuenta con mis datos, para comprar con carrito persistente, favoritos e historial. |
| **Criterios de Aceptación** | • Obliga los campos nombre, email, password y direccion_envio; telefono es opcional.<br>• Retorna estado 201 con objeto usuario (rol cliente), access_token (24h) y refresh_token (30d).<br>• Retorna 409 con código: EMAIL_DUPLICADO si el email ya está registrado.<br>• Retorna 400 si falta algún campo obligatorio. |
| **Mapeo Técnico** | Endpoint POST /api/auth/registro. En el frontend, RegisterView consume el resultado y muestra el error de email duplicado bajo el campo correspondiente. |

**HU-02 — Inicio de sesión**

| | |
|---|---|
| **Rol / Actor** | Invitado / Cliente |
| **Narrativa** | Como usuario registrado, quiero entrar con mi email y contraseña, para acceder a mi cuenta, carrito y pedidos. |
| **Criterios de Aceptación** | • Retorna 400 sin email/password; retorna 401 si las credenciales son inválidas.<br>• Sin 2FA activo: retorna 200 con los tokens de sesión; con 2FA activo: retorna requiere_2fa junto a un token temporal (10 min).<br>• Tras un login exitoso, se cargan automáticamente el perfil, carrito y favoritos. |
| **Mapeo Técnico** | Endpoint POST /api/auth/login. En el frontend, LoginView y authStore.login. |

**HU-03 — Verificación en dos pasos (2FA)**

| | |
|---|---|
| **Rol / Actor** | Cliente / Administrador |
| **Narrativa** | Como usuario, quiero activar o desactivar la verificación en dos pasos, para proteger mi cuenta con un código de 6 dígitos enviado a mi correo. |
| **Criterios de Aceptación** | • El switch en el perfil refleja el estado actual (two_factor_enabled).<br>• Activar/desactivar requiere confirmación y muestra un aviso visual.<br>• Con 2FA activo, el login devuelve un token temporal (10 min) y envía el código por email.<br>• El código expira a los 5 minutos y permite 5 intentos fallidos. |
| **Mapeo Técnico** | Endpoints POST /api/auth/activar-2fa y /desactivar-2fa. Sección "Seguridad" en ProfileView.vue. authStore.activar2FA() / desactivar2FA(). |

**HU-04 — Recuperación de contraseña con código**

| | |
|---|---|
| **Rol / Actor** | Cliente / Administrador |
| **Narrativa** | Como usuario que olvidó su contraseña, quiero recibir un código de 6 dígitos por correo, para restablecerla sin depender de un enlace. |
| **Criterios de Aceptación** | • El flujo es de 3 pasos: email → código → nueva contraseña.<br>• El código expira a los 15 minutos y permite 5 intentos fallidos.<br>• Se puede reenviar el código cada 60 segundos.<br>• El token de restablecimiento (de un solo uso) se emite sólo tras verificar el código.<br>• Los enlaces antiguos de recuperación siguen funcionando. |
| **Mapeo Técnico** | Tabla codigo_recuperacion + modelo CodigoRecuperacion. Endpoints POST /api/auth/olvide-contrasena, /verificar-codigo-recuperacion, /reenviar-codigo-recuperacion. Asistente en ForgotPasswordView.vue. |

**HU-05 — Notificación visual de cierre de sesión**

| | |
|---|---|
| **Rol / Actor** | Cliente / Administrador |
| **Narrativa** | Como usuario, quiero ver una notificación cuando cierro sesión, para tener confirmación visual de la acción. |
| **Criterios de Aceptación** | • Al cerrar sesión manualmente se muestra un toast "Sesión cerrada exitosamente".<br>• Si la sesión expira por token vencido, el aviso es informativo ("Tu sesión expiró. Vuelve a iniciar sesión.").<br>• El toast aparece incluso si el logout es forzado internamente. |
| **Mapeo Técnico** | authStore.logoutNotice (objeto {tipo, mensaje}). App.vue tiene un watcher que muestra el toast. vue-toastification configurado globalmente. |

**HU-06 — Guardar varias direcciones en mi cuenta**

| | |
|---|---|
| **Rol / Actor** | Cliente registrado |
| **Narrativa** | Como cliente registrado, quiero guardar varias direcciones de envío en mi cuenta, para elegir la adecuada en cada pedido sin volver a escribirla. |
| **Criterios de Aceptación** | • Puedo agregar, editar y eliminar direcciones desde mi perfil.<br>• Cada dirección tiene un alias opcional, una dirección completa (mínimo 10 caracteres) y una marca de "principal".<br>• Solo puede haber una dirección principal por cuenta.<br>• Al marcar una dirección como principal, las demás dejan de serlo.<br>• Al eliminar la principal, se promueve automáticamente la más antigua. |
| **Mapeo Técnico** | Tabla usuario_direcciones + endpoints GET/POST/PUT/DELETE /api/usuario/direcciones. Sección de gestión en ProfileView.vue. |

**HU-07 — Guardar varios teléfonos en mi cuenta**

| | |
|---|---|
| **Rol / Actor** | Cliente registrado |
| **Narrativa** | Como cliente registrado, quiero guardar varios teléfonos en mi cuenta, para tener un principal y otros de referencia. |
| **Criterios de Aceptación** | • Puedo agregar, editar y eliminar teléfonos (mínimo 8 dígitos).<br>• Solo puede haber un teléfono principal.<br>• Al marcar uno como principal, el anterior deja de serlo. |
| **Mapeo Técnico** | Tabla usuario_telefonos + endpoints GET/POST/PUT/DELETE /api/usuario/telefonos. |

**HU-08 — Elegir dirección y teléfono en el checkout**

| | |
|---|---|
| **Rol / Actor** | Cliente autenticado |
| **Narrativa** | Como cliente autenticado, quiero elegir en el checkout una dirección y un teléfono que ya tengo guardados (o usar otros nuevos y guardarlos), para completar la compra más rápido. |
| **Criterios de Aceptación** | • Selector de direcciones registradas + opción "Usar otra dirección para este pedido".<br>• Si escribo una nueva dirección, puedo marcar "Guardar esta dirección en mi cuenta" (se guarda como secundaria si ya había principal).<br>• Lo mismo para teléfonos, con un campo adicional "Teléfono de referencia" (opcional).<br>• El pedido guarda el snapshot de la dirección, del teléfono principal y del segundo teléfono.<br>• Para invitados, el flujo actual se mantiene sin cambios. |
| **Mapeo Técnico** | CheckoutView.vue (selector + checkboxes + telefono_2). POST /api/pedidos acepta direccion_id, telefono_id/telefono, telefono_2, guardar_direccion, guardar_telefono. |

**HU-09 — Ver el costo de envío en el carrito**

| | |
|---|---|
| **Rol / Actor** | Cliente |
| **Narrativa** | Como cliente, quiero ver en el carrito el costo de envío y el total real del pedido, para saber cuánto voy a pagar antes de confirmar. |
| **Criterios de Aceptación** | • El carrito muestra Subtotal, Envío y Total.<br>• El envío es un costo fijo (Q30 por defecto) salvo que el subtotal supere un mínimo configurable (Q200 por defecto), en cuyo caso es gratis.<br>• Cuando el envío es gratis, se muestra el badge "Gratis".<br>• Se muestra cuánto falta para obtener envío gratis (si aplica).<br>• El mismo desglose aparece en el checkout y en el detalle del pedido. |
| **Mapeo Técnico** | Tabla configuracion_tienda (key-value) con costo_envio y minimo_envio_gratis. pedidos agrega subtotal_productos y costo_envio. CheckoutView.vue y OrderDetailView.vue. |

**HU-10 — Configurar la política de envío (admin)**

| | |
|---|---|
| **Rol / Actor** | Administrador |
| **Narrativa** | Como administrador, quiero poder cambiar el costo de envío y el mínimo para envío gratis, para ajustar la política comercial sin tocar código. |
| **Criterios de Aceptación** | • En /admin/configuracion hay un formulario para editar costo_envio y minimo_envio_gratis.<br>• Solo acepta valores numéricos ≥ 0.<br>• Los valores quedan guardados y se reflejan en el checkout inmediatamente.<br>• GET /api/configuracion (admin) devuelve todas las claves. |
| **Mapeo Técnico** | Endpoints GET/PUT /api/configuracion. Panel en AdminDashboardView.vue. |

**HU-11 — Activar o desactivar el envío gratis**

| | |
|---|---|
| **Rol / Actor** | Administrador |
| **Narrativa** | Como administrador, quiero poder activar o desactivar la promoción de envío gratis, para encenderla o apagarla sin tocar el mínimo configurado. |
| **Criterios de Aceptación** | • En /admin/configuracion hay un switch "Envío gratis por monto mínimo".<br>• Con el switch apagado, el envío se cobra siempre y el cliente no ve ni el contador ni el aviso de envío gratis.<br>• El desglose del pedido (costo_envio, envio_gratis) refleja la configuración vigente al momento de la compra. |
| **Mapeo Técnico** | configuracion_tienda agrega la clave envio_gratis_activo ('true'/'false'). POST /api/pedidos la consulta para decidir. |

**HU-12 — Descuento por producto con rango de fechas**

| | |
|---|---|
| **Rol / Actor** | Administrador |
| **Narrativa** | Como administrador, quiero poder asignar a un producto un descuento por porcentaje con fecha de inicio y fin, para hacer promociones por tiempo limitado. |
| **Criterios de Aceptación** | • Puedo crear, editar, activar/desactivar y eliminar descuentos.<br>• El porcentaje está entre 1 y 100; la fecha final debe ser posterior a la inicial.<br>• Solo aplica el descuento vigente (activo y dentro del rango).<br>• Si dos descuentos se solapan, se aplica el de mayor porcentaje (y en empate, el que termina antes). |
| **Mapeo Técnico** | Tabla producto_descuentos + endpoints GET/POST/PUT/DELETE /api/products/<id>/descuentos. Gestión en AdminProductoEditarView.vue. |

**HU-13 — Precio con descuento en el catálogo**

| | |
|---|---|
| **Rol / Actor** | Cliente |
| **Narrativa** | Como cliente, quiero ver en el catálogo y en el detalle del producto el precio original tachado y el precio con descuento, para saber cuánto ahorro si compro ahora. |
| **Criterios de Aceptación** | • El producto muestra precio_descuento, tiene_descuento y porcentaje_descuento en su JSON.<br>• El campo precio conserva el precio de catálogo (compatibilidad).<br>• En el carrito, el ítem muestra el precio original tachado y el subtotal con el descuento aplicado. |
| **Mapeo Técnico** | Producto.to_dict() agrega los campos. El cálculo del descuento vigente se realiza en el modelo para mantener la consistencia. |

**HU-14 — Guardar el precio real pagado en el pedido**

| | |
|---|---|
| **Rol / Actor** | Administrador |
| **Narrativa** | Como administrador, quiero que el detalle del pedido guarde el precio real pagado y el porcentaje de descuento aplicado, para tener trazabilidad de las ventas con descuento. |
| **Criterios de Aceptación** | • detalle_pedidos guarda precio_unitario (de catálogo), precio_real (cobrado) y descuento_porcentaje.<br>• subtotal = precio_real * cantidad.<br>• El cálculo vive en un solo lugar (modelo), por lo que el precio del catálogo y el cobrado no divergen. |
| **Mapeo Técnico** | detalle_pedidos agrega precio_real y descuento_porcentaje. POST /api/pedidos los calcula y guarda. |

**HU-15 — Reportes coherentes y sin errores**

| | |
|---|---|
| **Rol / Actor** | Administrador |
| **Narrativa** | Como administrador, quiero que las exportaciones a Excel muestren de qué fecha a qué fecha es la información y que no den error al descargar, para poder analizar las ventas por periodo. |
| **Criterios de Aceptación** | • Los 7 reportes incluyen siempre un subtítulo con el periodo: Periodo: YYYY-MM-DD al YYYY-MM-DD (o "todo el historial" si no hay rango).<br>• Los nombres de archivo incluyen las fechas del filtro.<br>• ventas-por-mes aplica el filtro desde/hasta.<br>• Las filas vacías o con None no rompen el merge.<br>• Todos los endpoints con excel=1 descargan sin error. |
| **Mapeo Técnico** | routes/reportes.py: usar openpyxl.utils.get_column_letter en vez de chr(64 + len(headers)). Incluir subtítulo de periodo. Aplicar filtro en ventas-por-mes. |

**HU-16 — Gráficos en el panel de reportes**

| | |
|---|---|
| **Rol / Actor** | Administrador |
| **Narrativa** | Como administrador, quiero ver gráficos de las ventas en el panel de reportes, para detectar tendencias de un vistazo. |
| **Criterios de Aceptación** | • Hay 5 gráficos funcionales: ventas por mes, ganancias por mes, productos más vendidos, clientes top y ventas con descuento.<br>• Los gráficos respetan el rango de fechas del filtro global.<br>• La librería de charts (chart.js + vue-chartjs) está en package.json y package-lock.json (build autónomo). |
| **Mapeo Técnico** | AdminReportesView.vue + chart.js/vue-chartjs. Los gráficos consumen los endpoints existentes. |

**HU-17 — Reporte de ventas con descuento**

| | |
|---|---|
| **Rol / Actor** | Administrador |
| **Narrativa** | Como administrador, quiero ver un reporte de ventas con descuento que me diga si las ventas incrementaron, para evaluar si las promociones funcionan. |
| **Criterios de Aceptación** | • Por producto: unidades, unidades con descuento, ingresos con descuento, ingresos sin descuento, ahorro total y descuento promedio.<br>• Totales del periodo y comparación con el periodo anterior equivalente (misma cantidad de días) con la variación porcentual de ingresos y unidades.<br>• Soporta excel=1. |
| **Mapeo Técnico** | Nuevo endpoint GET /api/reportes/ventas-con-descuento. Tarjeta/gráfico en AdminReportesView.vue. |

**HU-18 — Galería de imágenes visible al editar un producto**

| | |
|---|---|
| **Rol / Actor** | Administrador |
| **Narrativa** | Como administrador, quiero ver las imágenes existentes y las nuevas antes de subirlas al editar un producto, para no creer que no tiene imágenes. |
| **Criterios de Aceptación** | • Al abrir la edición, se muestran las imágenes ya guardadas (de la galería).<br>• Los productos creados antes de la galería (solo con imagen_url) muestran su imagen principal como "Imagen principal del producto".<br>• Se muestra un indicador de carga para evitar parpadeos.<br>• Si el endpoint de imágenes falla, se avisa en la interfaz y se usa el respaldo (prod.imagenes).<br>• Las nuevas imágenes se previsualizan antes de subir. |
| **Mapeo Técnico** | AdminProductoEditarView.vue: cargandoGaleria, errorGaleria, imagenLegacy, limpiarNuevos. |

**HU-19 — Panel de administración agrupado por secciones**

| | |
|---|---|
| **Rol / Actor** | Administrador |
| **Narrativa** | Como administrador, quiero que el panel /admin agrupe las secciones por título, para encontrar rápido lo que necesito. |
| **Criterios de Aceptación** | • Tres grupos: Productos (productos, categorías, marcas, inventario), Pedidos (pedidos) y Reportes (reseñas, reportes, configuración).<br>• Se elimina la alerta "Usá las tarjetas para navegar a cada sección".<br>• Cada tarjeta sigue llevando a su página dedicada. |
| **Mapeo Técnico** | AdminDashboardView.vue reorganizado. |

**HU-20 — Buscador en la barra superior**

| | |
|---|---|
| **Rol / Actor** | Cliente |
| **Narrativa** | Como cliente, quiero poder buscar productos desde la barra superior en cualquier página, para encontrar lo que busco sin ir primero al catálogo. |
| **Criterios de Aceptación** | • El buscador está centrado entre la marca y los iconos.<br>• Escribe la búsqueda en la URL (/catalogo?buscar=...) y el catálogo la aplica al instante.<br>• En móvil se oculta y se despliega con el botón de lupa.<br>• El icono de lupa se oculta mientras el campo está visible.<br>• El botón "Limpiar todo" del catálogo vacía el texto del buscador. |
| **Mapeo Técnico** | App.vue navbar + CatalogView.vue. |

**HU-21 — Iconos del carrito y del perfil**

| | |
|---|---|
| **Rol / Actor** | Cliente |
| **Narrativa** | Como cliente, quiero que los iconos de notificaciones, carrito y perfil estén en este orden (de izquierda a derecha), para encontrarlos rápido. |
| **Criterios de Aceptación** | • El orden es: notificaciones, carrito, perfil.<br>• El carrito se muestra a clientes e invitados (funciona con token de invitado).<br>• Los administradores no ven el icono del carrito. |
| **Mapeo Técnico** | App.vue navbar. |

**HU-22 — Menú del usuario simplificado (admin)**

| | |
|---|---|
| **Rol / Actor** | Administrador |
| **Narrativa** | Como administrador, quiero que el menú de mi avatar tenga solo Mi perfil, Panel de administración y Cerrar sesión, para no duplicar las secciones que ya están en /admin. |
| **Criterios de Aceptación** | • El dropdown del administrador tiene exactamente esas tres opciones.<br>• El dropdown del cliente mantiene Mi perfil, Mis pedidos, Mis favoritos y Cerrar sesión. |
| **Mapeo Técnico** | App.vue dropdown condicional por isAdmin. |

**HU-23 — Filtros del catálogo en una mini barra**

| | |
|---|---|
| **Rol / Actor** | Cliente |
| **Narrativa** | Como cliente, quiero que los filtros del catálogo estén en una barra bajo la barra principal, para ganar espacio para los productos. |
| **Criterios de Aceptación** | • La barra tiene los botones "Ordenar por", "Marca" y "Categoría" (cada uno abre un desplegable con las opciones).<br>• Se elimina la columna lateral de filtros.<br>• En móvil, los tres botones caben en una sola línea.<br>• Los desplegables quedan por encima de la barra.<br>• Las etiquetas de filtros activos se conservan debajo de la barra. |
| **Mapeo Técnico** | CatalogView.vue. |

**HU-24 — Cuadrícula de productos**

| | |
|---|---|
| **Rol / Actor** | Cliente |
| **Narrativa** | Como cliente, quiero ver los productos en una cuadrícula de 4 por fila en escritorio y 2 en móvil, para aprovechar mejor el espacio. |
| **Criterios de Aceptación** | • Escritorio: 4 productos por fila.<br>• Tablet: 3 por fila.<br>• Móvil: 2 por fila.<br>• La tarjeta se adapta al ancho (precio y botón "Añadir" no se aprietan). |
| **Mapeo Técnico** | CatalogView.vue (grid de Bootstrap). |

**HU-25 — Marca en dos líneas en móvil**

| | |
|---|---|
| **Rol / Actor** | Cliente en móvil |
| **Narrativa** | Como cliente en móvil, quiero que el texto "KeiBeauty" se muestre más compacto, para dejar espacio para el buscador y los iconos. |
| **Criterios de Aceptación** | • En móvil, la marca se muestra en dos líneas: "Kei" y "Beauty".<br>• El texto es más pequeño y no se trunca. |
| **Mapeo Técnico** | App.vue navbar (CSS responsive). |

**HU-26 — Barra superior estable en móvil**

| | |
|---|---|
| **Rol / Actor** | Cliente en móvil |
| **Narrativa** | Como cliente en móvil, quiero que la barra superior no cambie de altura al abrir la búsqueda, para que la página no salte. |
| **Criterios de Aceptación** | • La altura del navbar es la misma con la búsqueda abierta o cerrada (64 px).<br>• El desplegable de la barra de filtros queda por encima de la barra al abrirse. |
| **Mapeo Técnico** | App.vue navbar (CSS min-height). |

**HU-27 — "Sobre nosotros" en el footer**

| | |
|---|---|
| **Rol / Actor** | Cliente |
| **Narrativa** | Como cliente, quiero encontrar el enlace a "Sobre nosotros" en el pie de página, para conocer la tienda sin ocupar espacio en la barra principal. |
| **Criterios de Aceptación** | • El navbar ya no tiene el enlace "Sobre nosotros".<br>• El footer incluye el enlace "Sobre nosotros" junto a "Catálogo". |
| **Mapeo Técnico** | App.vue navbar/footer. |

**HU-28 — Despliegue en AWS con CI/CD**

| | |
|---|---|
| **Rol / Actor** | Equipo de desarrollo |
| **Narrativa** | Como equipo, queremos desplegar el frontend en S3+CloudFront, el backend en EC2 y la base de datos en RDS, con CI/CD por GitHub Actions, para tener un entorno de producción estable. |
| **Criterios de Aceptación** | • Frontend en S3 + CloudFront (HTTPS, invalidación /*).<br>• Backend en EC2 (Ubuntu) con venv, servicio systemd keibeauty + nginx (TLS con Let's Encrypt).<br>• Base de datos en RDS PostgreSQL (solo accesible desde la red interna).<br>• GitHub Actions dispara en main: backend aplica flask db upgrade (fail-fast) y reinicia servicios; frontend hace npm run build + s3 sync + invalidación.<br>• develop es la rama de integración; main se actualiza cuando el trabajo está listo. |
| **Mapeo Técnico** | .github/workflows/deploy.yml de cada repo. config.py con CORS para CloudFront (https://d2agaijw0kvv8d.cloudfront.net). |

# Casos de Uso

**CU-01 — Registrar Usuario**

| | |
|---|---|
| **Actor(es)** | Cliente Invitado |
| **Descripción** | Creación de cuenta de cliente con datos de contacto y dirección de envío. |
| **Precondiciones** | No tener sesión activa. |
| **Flujo Básico** | 1. El invitado abre/registro y completa nombre, email, password, teléfono y dirección.<br>2. El frontend envía POST /api/auth/registro.<br>3. El backend valida campos obligatorios, rechaza email duplicado (409), aplica hash a la password y crea el usuario con rol cliente.<br>4. Devuelve estado 201 con el objeto usuario, access_token (24h) y refresh_token (30d).<br>5. El frontend almacena la sesión, carga perfil/carrito/favoritos y redirige a /catalogo. |
| **Flujos Alternativos / Excepciones** | Estado 400 por campo obligatorio faltante; 409 si el email ya está registrado; si el cliente falla por red/timeout, el mensaje indica que puede reintentar. |
| **HU Asociada** | HU-01 |

**CU-02 — Iniciar Sesión**

| | |
|---|---|
| **Actor(es)** | Cliente Invitado / Registrado |
| **Descripción** | Autenticación mediante email y contraseña, con derivación opcional a flujo 2FA. |
| **Precondiciones** | Cuenta de usuario existente; no tener sesión activa. |
| **Flujo Básico** | 1. Ingresa credenciales en /login → ejecuta POST /api/auth/login.<br>2. Sin 2FA activo: responde 200 con tokens y objeto usuario; el frontend hidrata la sesión.<br>3. Con 2FA activo: responde 200 con indicador requiere_2fa, email y token temporal (10 min); continúa en CU-03. |
| **Flujos Alternativos / Excepciones** | Estado 400 por campos vacíos; estado 401 por credenciales inválidas. |
| **HU Asociada** | HU-02 |

**CU-03 — Verificar Código 2FA**

| | |
|---|---|
| **Actor(es)** | Cliente / Administrador |
| **Descripción** | Validación del código de 6 dígitos durante el login. |
| **Precondiciones** | Haber iniciado sesión con 2FA activo (token temporal). |
| **Flujo Básico** | 1. El usuario ingresa el código en /verificar-2fa.<br>2. El frontend envía POST /api/auth/verificar-2fa con el token temporal.<br>3. El backend valida el código; si es correcto, devuelve los tokens reales.<br>4. El frontend hidrata la sesión y carga carrito/favoritos. |
| **Flujos Alternativos / Excepciones** | Estado 400 por código inválido, expirado o demasiados intentos. POST /api/auth/reenviar-codigo-2fa con rate limit de 60 s. POST /api/auth/cancelar-login invalida los códigos pendientes. |
| **HU Asociada** | HU-03 |

**CU-04 — Restablecer Contraseña con Código**

| | |
|---|---|
| **Actor(es)** | Cliente / Administrador |
| **Descripción** | Restablecimiento de contraseña mediante código de 6 dígitos. |
| **Precondiciones** | Cuenta de usuario existente. |
| **Flujo Básico** | 1. El usuario abre /olvide-contrasena e ingresa su email → POST /api/auth/olvide-contrasena.<br>2. El backend genera un código de 6 dígitos, lo hashea en codigo_recuperacion y lo envía por email (la respuesta es siempre 200).<br>3. El usuario ingresa el código → POST /api/auth/verificar-codigo-recuperacion.<br>4. Si el código es válido, el backend devuelve un token de un solo uso.<br>5. El usuario define la nueva contraseña → POST /api/auth/reestablecer-contrasena.<br>6. El backend actualiza la password y marca el token como usado. |
| **Flujos Alternativos / Excepciones** | Código incorrecto (400) con contador de intentos; código expirado (400); reenvío con rate limit de 60 s (429); token inválido/expirado (400). |
| **HU Asociada** | HU-04 |

**CU-05 — Cerrar Sesión**

| | |
|---|---|
| **Actor(es)** | Cliente / Administrador |
| **Descripción** | Cierre de la sesión con aviso visual. |
| **Precondiciones** | Sesión activa. |
| **Flujo Básico** | 1. El usuario hace clic en "Cerrar sesión" en el menú de avatar.<br>2. authStore.logout() limpia tokens, usuario y carrito/favoritos locales.<br>3. Se establece logoutNotice → el watcher de App.vue muestra el toast.<br>4. Se redirige a /. |
| **Flujos Alternativos / Excepciones** | Si el token expiró al intentar refrescar, el logout se dispara internamente con motivo "Tu sesión expiró...". |
| **HU Asociada** | HU-05 |

**CU-06 — Gestionar Direcciones y Teléfonos**

| | |
|---|---|
| **Actor(es)** | Cliente registrado |
| **Descripción** | CRUD de direcciones y teléfonos desde el perfil. |
| **Precondiciones** | Tener sesión activa. |
| **Flujo Básico** | 1. El cliente abre /perfil y va a la sección de direcciones/teléfonos.<br>2. Puede agregar (alias, dirección/teléfono, marcar principal), editar o eliminar.<br>3. El backend valida (dirección ≥10 chars, teléfono ≥8 dígitos), actualiza la tabla y sincroniza usuarios.direccion_envio/usuarios.telefono con la principal.<br>4. Al eliminar la principal, se promueve la siguiente. |
| **Flujos Alternativos / Excepciones** | Dirección con menos de 10 caracteres → 400. Alias mayor a 50 caracteres → 400. Teléfono con menos de 8 dígitos → 400. direccion_id inválido en pedido → 404. |
| **HU Asociada** | HU-06, HU-07 |

**CU-07 — Checkout con Dirección y Teléfonos Guardados**

| | |
|---|---|
| **Actor(es)** | Cliente / Invitado |
| **Descripción** | Selección o creación de dirección y teléfonos durante el checkout. |
| **Precondiciones** | Carrito con ítems. |
| **Flujo Básico** | 1. El cliente autenticado ve un selector de direcciones registradas + "Usar otra dirección".<br>2. Si elige una registrada, se envía direccion_id; si es nueva, se envía direccion_envio + guardar_direccion.<br>3. Lo mismo para teléfonos, con telefono_2 opcional.<br>4. El backend crea el pedido con el snapshot de dirección y teléfonos. |
| **Flujos Alternativos / Excepciones** | Invitado → flujo actual sin cambios (dirección y teléfono obligatorios). guardar_direccion=true con texto libre → crea dirección secundaria si ya había principal. |
| **HU Asociada** | HU-08 |

**CU-08 — Crear Pedido con Envío y Descuento**

| | |
|---|---|
| **Actor(es)** | Cliente / Invitado |
| **Descripción** | Creación de pedido con cálculo de subtotal, descuento y envío. |
| **Precondiciones** | Carrito con ítems. |
| **Flujo Básico** | 1. El frontend envía POST /api/pedidos (con direccion_id/direccion_envio, teléfono, etc.).<br>2. El backend calcula subtotal_productos, determina costo_envio según el mínimo configurado y guarda monto_total.<br>3. Guarda el snapshot de dirección, teléfonos, costo_envio y subtotal_productos en pedidos.<br>4. Devuelve el desglose en la respuesta. |
| **Flujos Alternativos / Excepciones** | Subtotal < mínimo → costo_envio = costo fijo. Envío gratis desactivado → costo_envio = costo fijo siempre. Invitado → flujo sin cambios. |
| **HU Asociada** | HU-09, HU-10, HU-11 |

**CU-09 — Asignar Descuento a un Producto**

| | |
|---|---|
| **Actor(es)** | Administrador |
| **Descripción** | CRUD de descuentos por producto con rango de fechas. |
| **Precondiciones** | Sesión de administrador. |
| **Flujo Básico** | 1. El admin abre la edición de un producto y gestiona sus descuentos.<br>2. Envía porcentaje (1-100), fecha de inicio y fecha de fin.<br>3. El backend valida y crea el descuento.<br>4. El producto muestra el precio con descuento en el catálogo mientras esté vigente. |
| **Flujos Alternativos / Excepciones** | Porcentaje fuera de rango → 400. fecha_fin <= fecha_inicio → 400. Dos descuentos solapados → se aplica el de mayor porcentaje. |
| **HU Asociada** | HU-12, HU-13 |

**CU-10 — Exportar Reportes a Excel**

| | |
|---|---|
| **Actor(es)** | Administrador |
| **Descripción** | Descarga de reportes en Excel con rango de fechas. |
| **Precondiciones** | Sesión de administrador. |
| **Flujo Básico** | 1. El admin abre /admin/reportes, elige fechas (desde/hasta) y hace clic en "Excel".<br>2. El frontend llama al endpoint con ?excel=1&desde=...&hasta=....<br>3. El backend genera el workbook con título, subtítulo de periodo y datos.<br>4. Se descarga el archivo .xlsx con el rango en el nombre. |
| **Flujos Alternativos / Excepciones** | Sin rango → "todo el historial". Fechas inválidas → 400. Filas vacías/None no rompen el merge. |
| **HU Asociada** | HU-15 |

**CU-11 — Desplegar en AWS**

| | |
|---|---|
| **Actor(es)** | Equipo de desarrollo |
| **Descripción** | Despliegue automatizado a AWS mediante GitHub Actions. |
| **Precondiciones** | Código mergeado a main en ambos repos. |
| **Flujo Básico** | 1. Se hace git merge develop → main y git push origin main en ambos repos.<br>2. GitHub Actions ejecuta el pipeline del backend: SSH al EC2, git pull, pip install, flask db upgrade, reinicio de servicios.<br>3. GitHub Actions ejecuta el pipeline del frontend: npm ci + npm run build + aws s3 sync + invalidación CloudFront.<br>4. Se verifican los endpoints con respuesta 200. |
| **Flujos Alternativos / Excepciones** | Si la migración falla, el deploy se detiene (set -e). El puerto 5000 no es accesible desde internet; la verificación se hace por dominio con TLS (443). |
| **HU Asociada** | HU-28 |

# Repositorios de GitHub

https://github.com/Jbrod5/KeiBeauty-frontend

https://github.com/Shejin12/KeiBeauty